

Java Programming Language helps in developing distributed applications as it is platform-independent. It was inspired by C++ and has many of its syntaxes.

It is a basic and robust programming language. One can build any type of application like enterprise application, web application, mobile application, and cloud-based application using Java.

Originally developed by James Gosling, it is the most popular programming language for enterprise applications.

Java is a high-level, general-purpose programming language developed by a small team, headed by James Gosling and Patrick Naughton at Sun Microsystems in 1991. It is currently owned by Oracle Corporation.

It was designed to be small, simple, and portable across platforms and operating systems.

It was initially developed for consumer devices. Later it has become a popular platform to develop enterprise applications.

Java 16 is the latest version of Java as of March 2021.

Java comes in various editions to cater to various types of application development needs.

Two key editions of Java are:

1. Java Standard Edition

2. Java Enterprise Edition

To learn Java programming fundamentals, we need Java SE.

Java SE (Standard Edition)

This edition consists of libraries for building core Java applications.

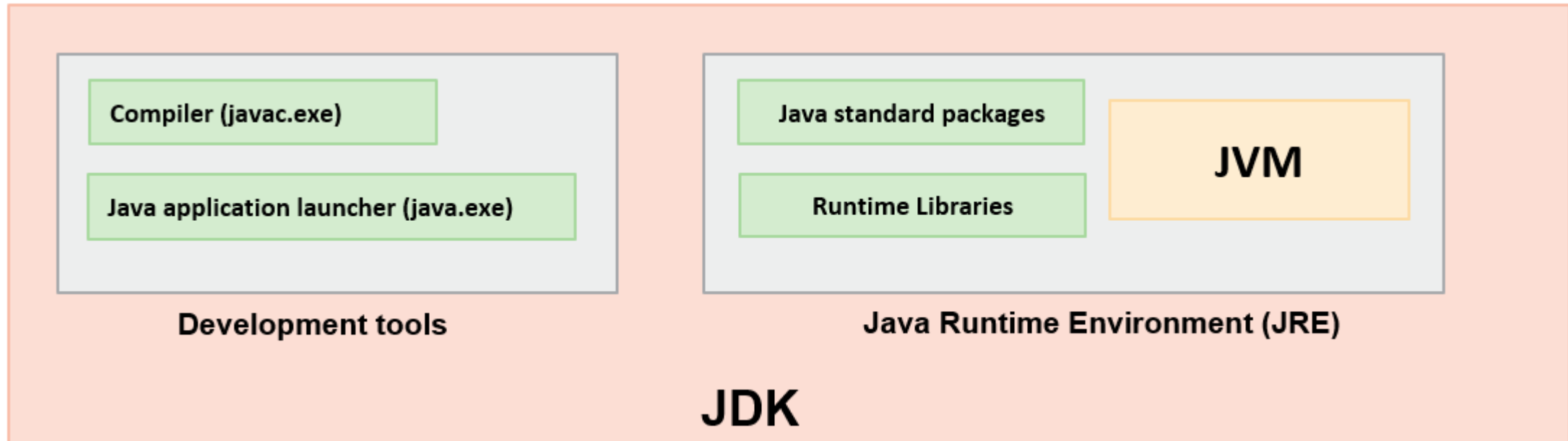
Java EE (Enterprise Edition)

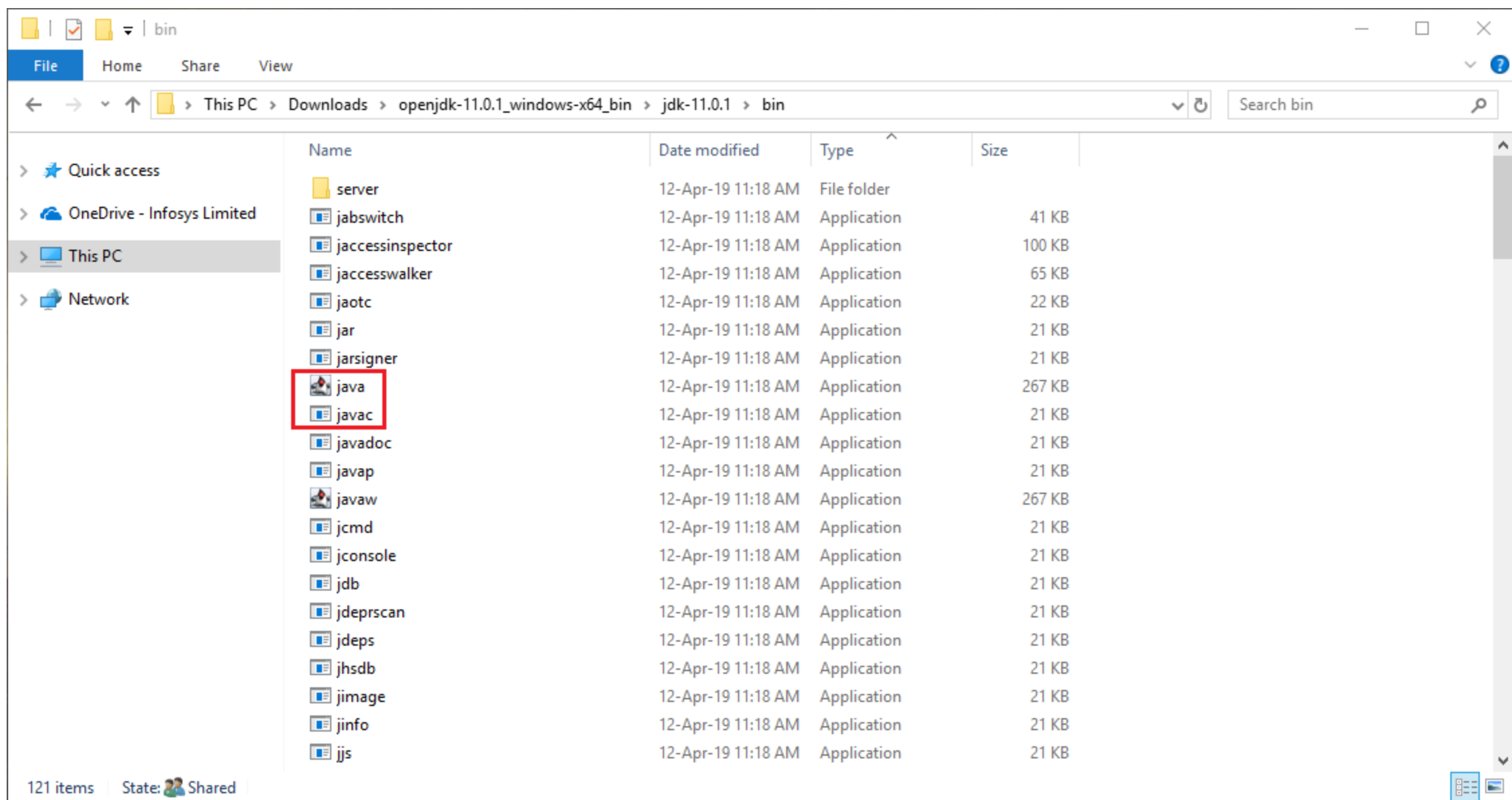
This edition consists of libraries for building distributed Java applications. These libraries depend on Java SE libraries

- Development Tools
- Java Runtime Environment (JRE)

Development tools include Java compiler (javac.exe) and Java interpreter (java.exe). These tools are required to prepare a Java application for execution.

Java Virtual Machine (JVM) executes a Java application using certain specific libraries and files provided by JRE. JVM is the actual runtime in which a Java application executes.





Why set Java path:

We know that the Java program is executed inside the JVM. So in any platform like windows, if we want to execute a Java program we should have JDK and we should tell the windows where the Java compiler is located. This is why we have an environment variable called "JAVA_HOME" that has to be allocated with the path pointing to the JDK's bin folder where we can find the Java compiler and all the other executables.

Now we can set a path in two ways.

- Using command prompt
- Editing the system's environment variable

Steps to set Java path using command prompt:

Step 1: Open command prompt

Step 2: Copy the path of your JDK folder

Step 3: Use the command to set path set JAVA_HOME=**paste your JDK path**

Step 4: Verify it by using the command: echo %JAVA_HOME%

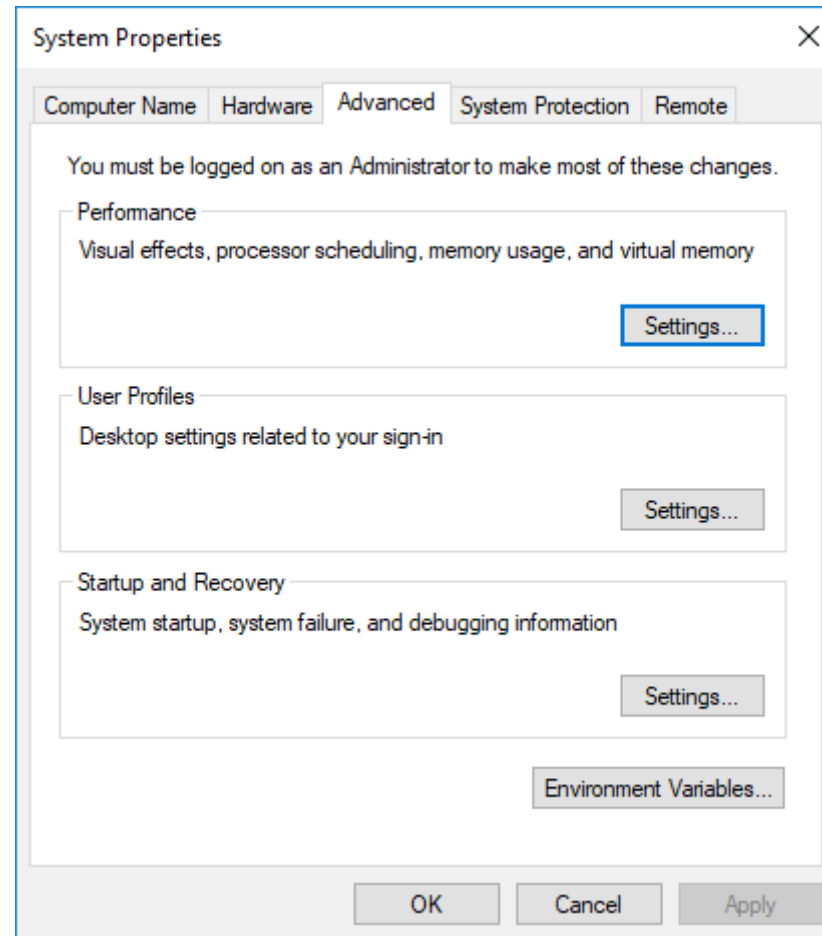
Step 5: Use the command: set PATH=%JAVA_HOME%\bin

Step 6: Verify it by using the command: echo %PATH%

Steps to set Java path using an environment variable:

Now to set the path, in the windows search type “env”. You will get an option “Edit Environment Variables” click on that.

In the “advanced” tab, click on the “Environment variables” option.



- Highlights:To understand how to develop and execute a Java program using the console.
- To understand how to develop a basic Java program.
- To understand how to execute a basic Java program.

The following prerequisite software and environment setup are needed for developing basic Java programs.

Java JDK

JDK

Eclipse IDE

Eclipse IDE for JEE Developers - 3.2.x above

Objective: To understand how to develop and execute a Java program using the console.

Step 1: Change your directory to the desktop by using the command: cd Desktop

Step 2: Create a folder using the command: mkdir **folder_name**

Step 3: Type the Java program in any text editor

Step 4: Save the file as HelloWorldApp.java in the folder that you've just created on the desktop.

Step 5: To compile the Java Program, type the command "javac HelloWorldApp.java" in the command prompt and press enter and if there are no compilation errors HelloWorldApp.class will be created in the same folder

Step 6: To execute the Java Program type "java HelloWorldApp" in the command prompt and press enter

Example to develop the first Java program using Eclipse IDE

Objective: To understand how to develop a basic Java program.

Step 1: Create a project as follows

- Launch the Eclipse IDE
- Access File option in the menu bar
- For project creation: File->New->Other->Java Project->name of the project(FirstJavaProgram) and click Finish
- The required project structure will be:

Step 2: Create a class `Main.java` as below

- Right click on src package-> New ->Class-> Provide class name (Main.java) and click Finish. Write below code in "Main.java" file

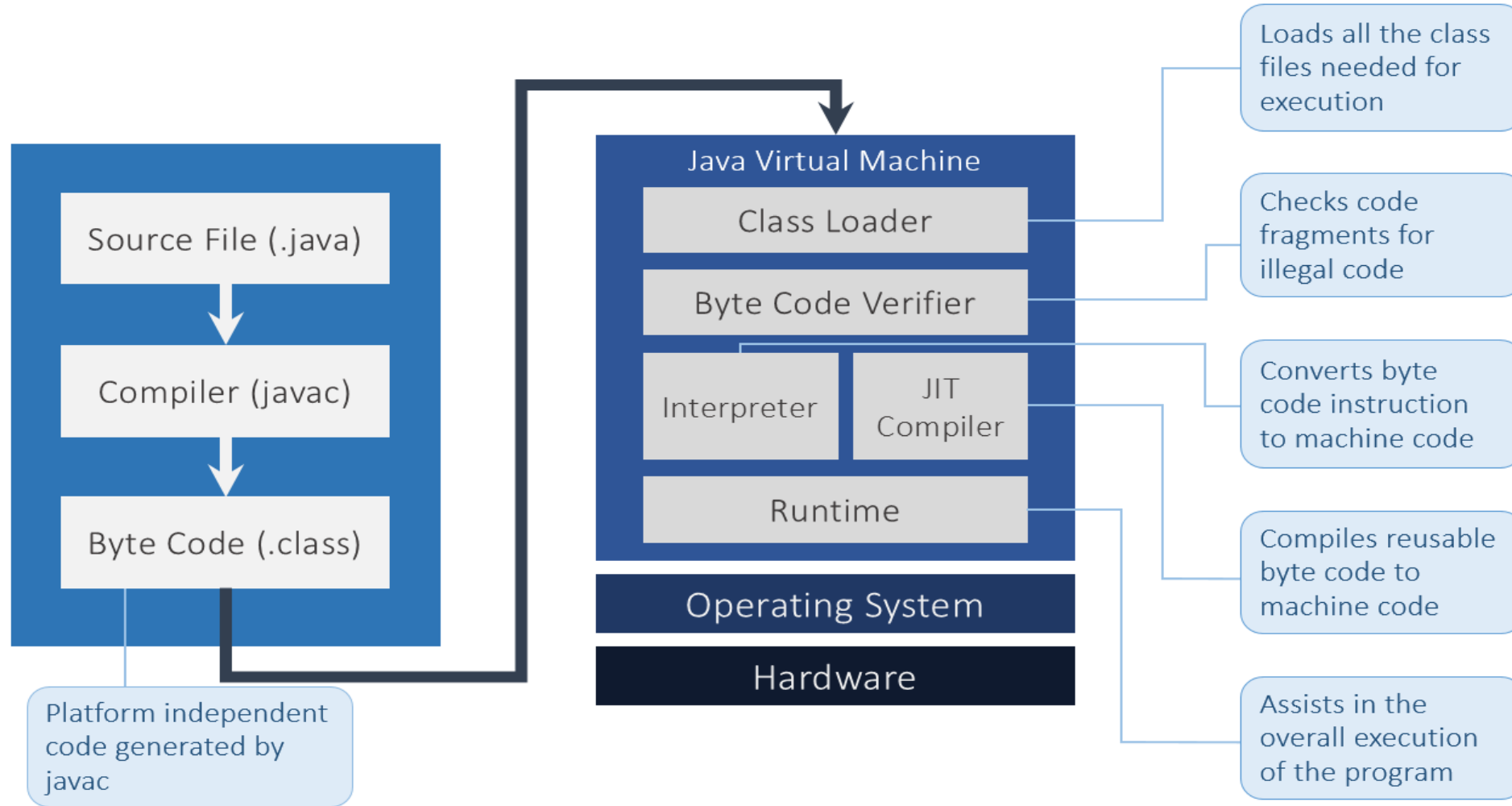
Step 3: Run the Java Program

- Right-click on project (FirstJavaProgram) -> Run as Java application.

Best Practices:

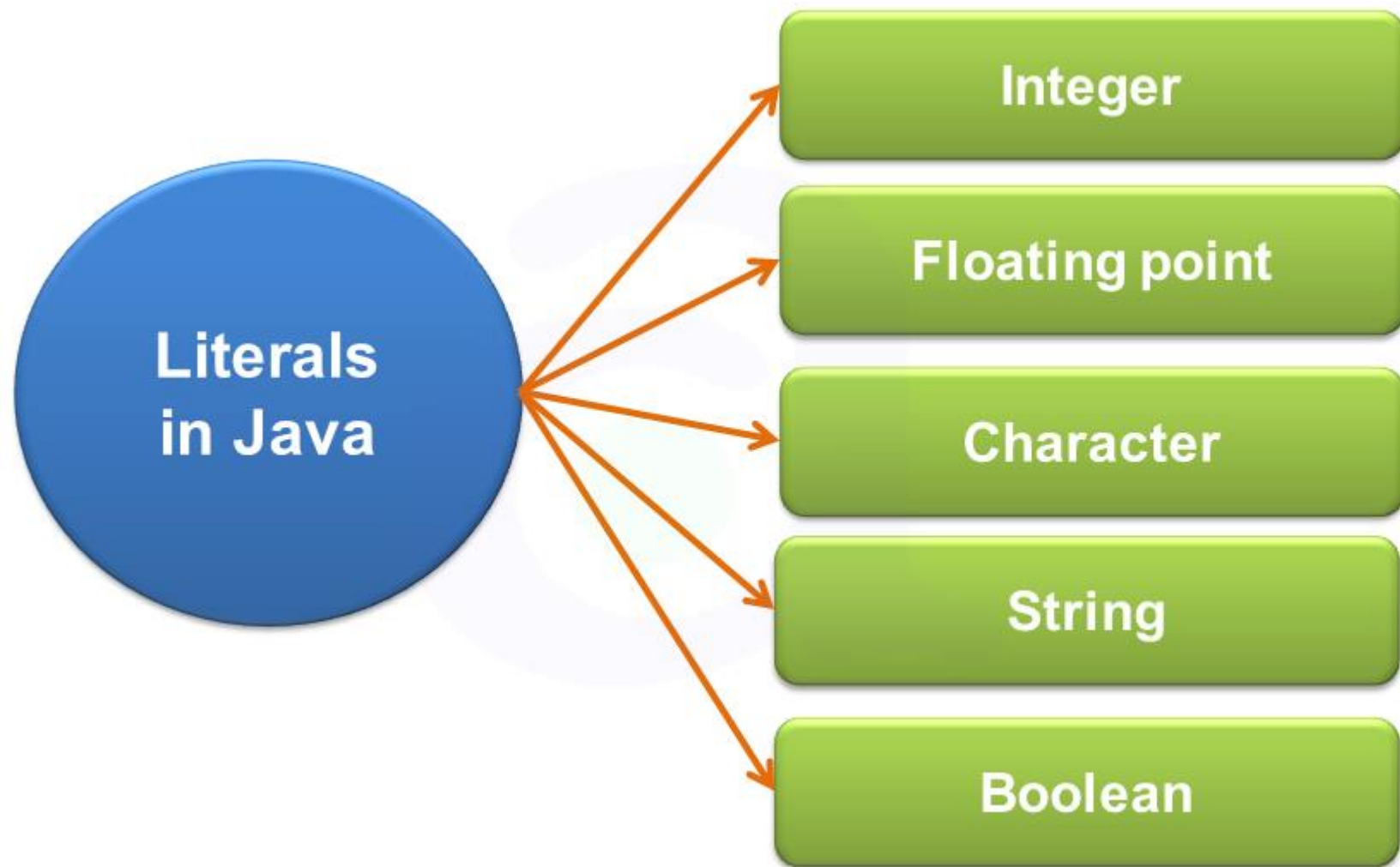
- The name of the class should start with an uppercase letter using Camel Case notation, like for example Employee
- The name of methods should start with a lowercase letter using CamelCase notation, like for example getName()

Now, we will go through the complete execution process of the Java application:

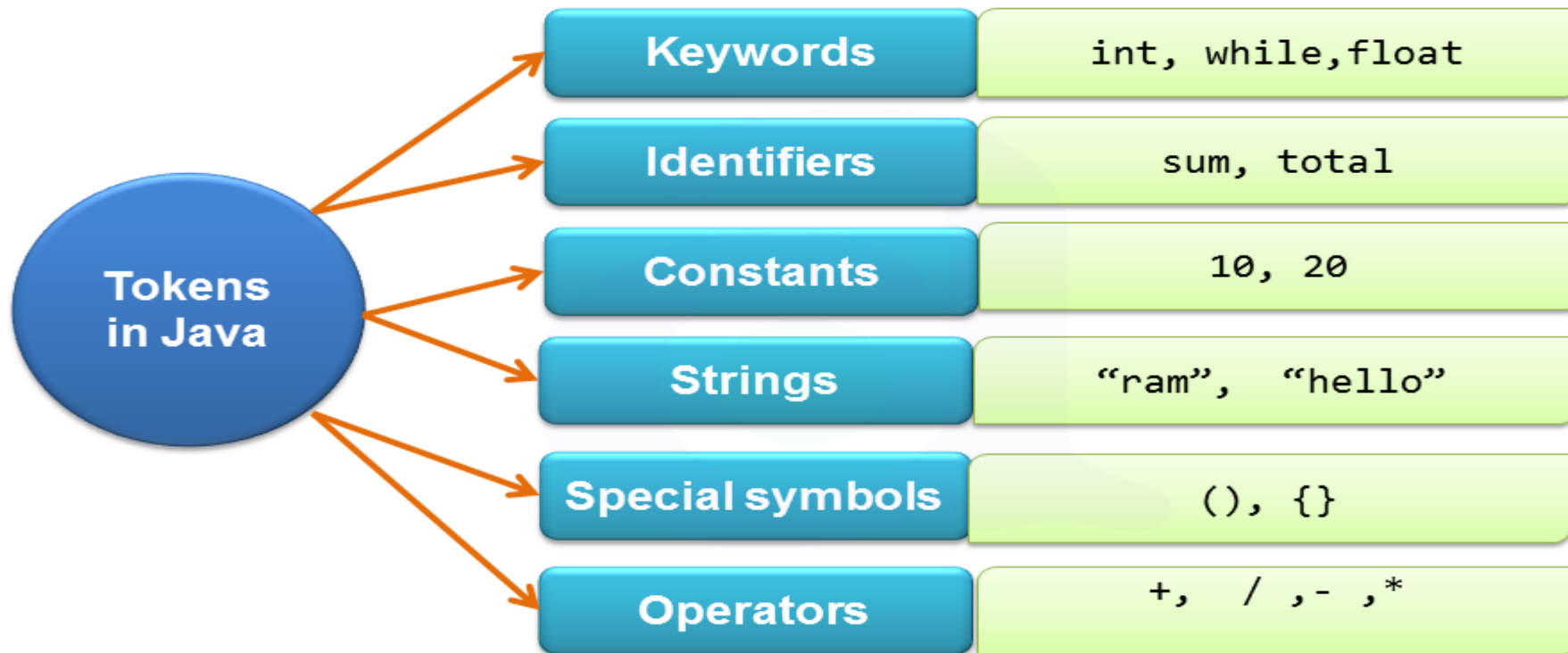


Features of Java. We will be exploring these features across this semester.





The compiler breaks lines into pieces of text known as Java tokens. Java supported Java tokens include keywords, variables, constants, special characters, operations and so on. The compiler scans the text in your source code when you compile a program, and extracts individual Java tokens



Every programming language has a set of **keywords** which are reserved words having a predefined meaning. Each keyword represents a specific functionality in the language.

You must have observed, a program has several components and blocks. It has words as well. Some of these words are reserved and have a special meaning in Java, e.g., class, public, void, for, int, static. These words are known as keywords. Currently, Java has 50 keywords.

Some of the keywords in Java language are listed below:

Keywords		
abstract	else	return
break	final	static
case	for	switch
class	if	throw
continue	import	try
default	interface	this
do	new	while

in Java, an **identifier** is the name given to a variable, method or class to uniquely identify it.

Not only keywords, but there can be user-defined words to identify different components of a program, e.g., Account, main known as Identifiers. These words are known as Identifiers. Identifiers are names used for identifying components of a program like classes, methods, interfaces, enums, and variables. After defining once, we can use them again to identify corresponding components in the program.

In Java, following rules apply to the identifier name:

- It can contain alphanumeric characters([a-z], [A-Z], [0-9]), dollar sign (\$), underscore (_)
- It should not start with a digit ([0-9])
- It should not have spaces
- It should not be a Java keyword
- It is case-sensitive
- It has no length restrictions



Here, the coffee cups are without any labels making it difficult to identify the companies



Here, the coffee cups have labels which help to identify the companies they belong to

Every program needs data and a mechanism to store this data.

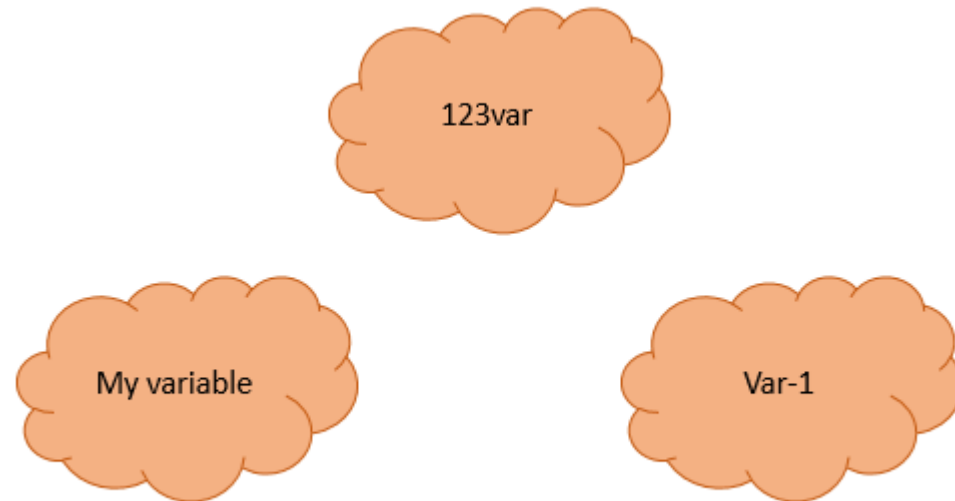
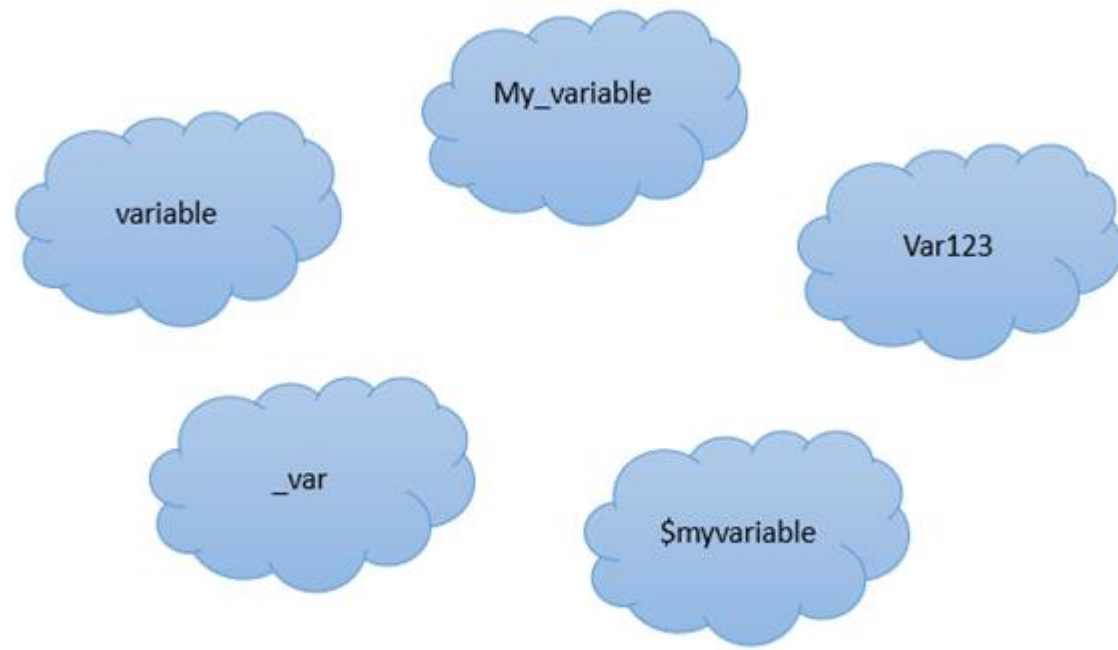
Data is stored in **variables**.

A variable is a named memory location which holds some value.

The value stored in a variable can vary during the execution of the program.

Observe the below code to understand how data is stored in variables.

```
public static void main(String[] args) {  
    int discount = 10; // discount is a variable  
    double totalPrice = 200; // totalPrice is a variable  
    double priceAfterDiscount = totalPrice * (1 - ((double) discount / 100)); //  
    priceAfterDiscount is a variable  
    System.out.println("Customer has paid a bill of amount: "+ priceAfterDiscount);  
}
```

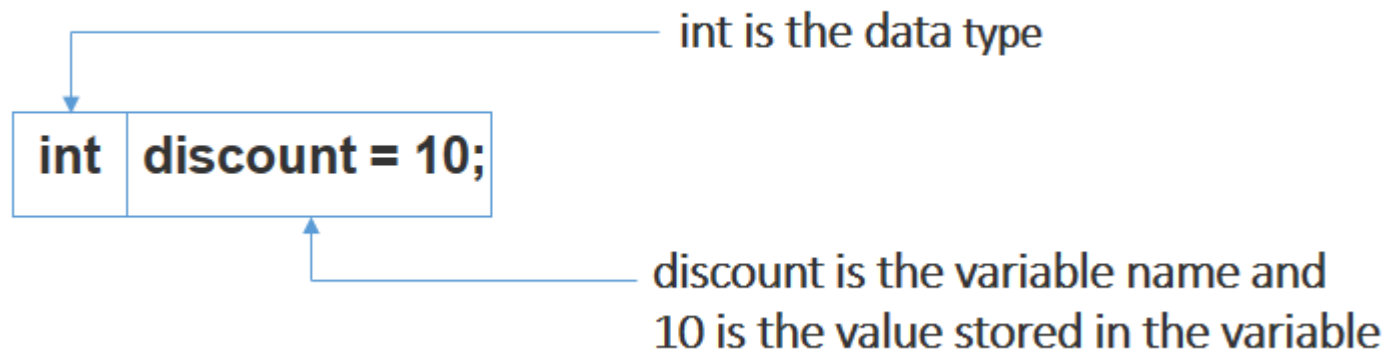


You can declare variables in Java as follows:

<data type> <variableName>

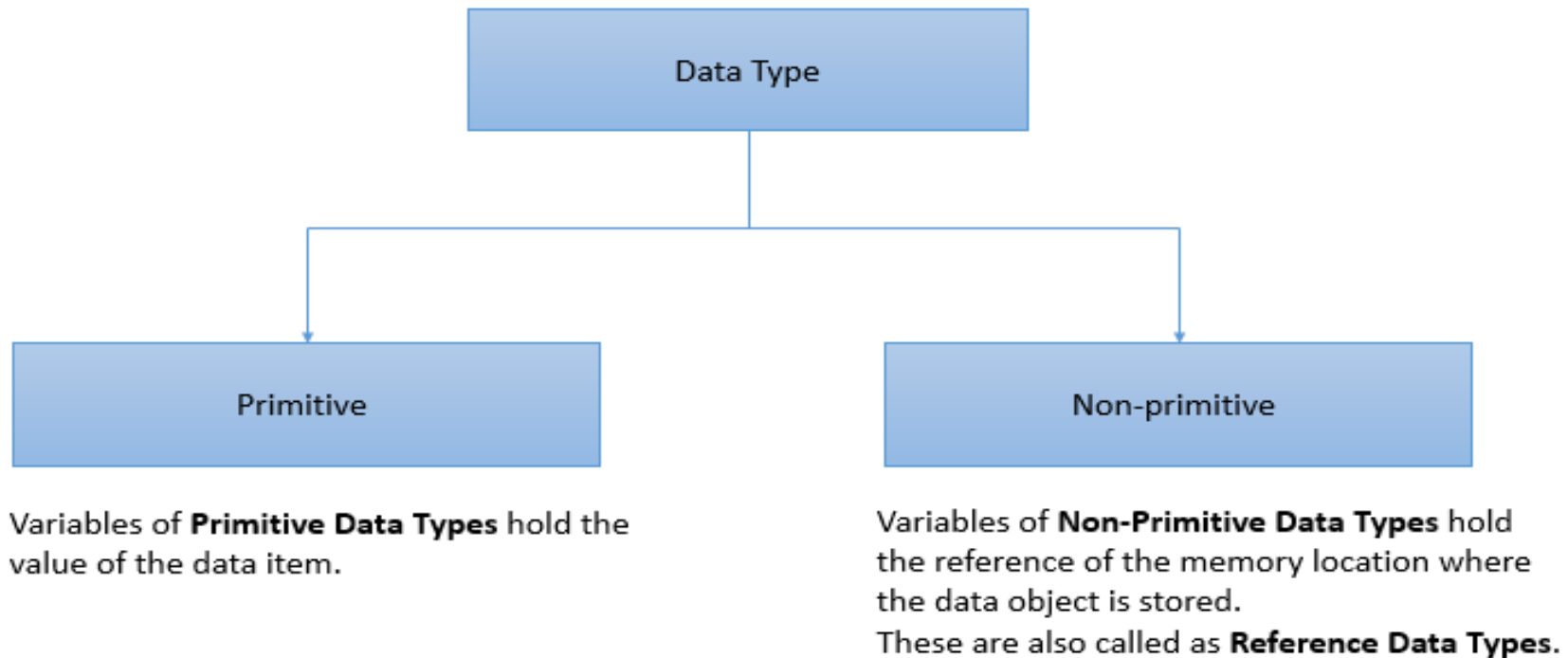
OR

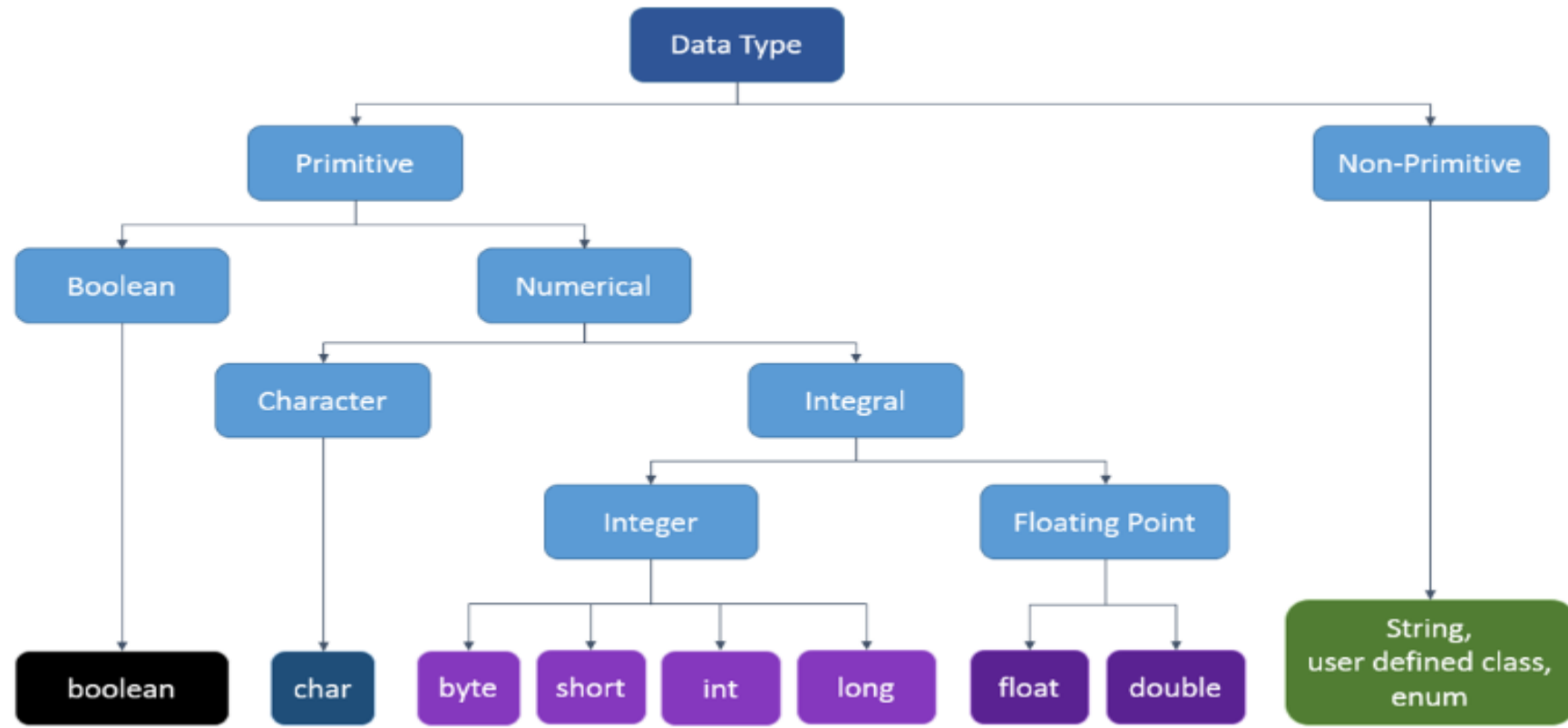
<data type> <variableName> = <value>



Data type defines the type of data that can be stored in the variable and the memory required by it.

There are two types of data types in Java:





Primitive data types are the basic data types defined in Java. There are 8 primitive data types as shown in the below table.

Data type	Description	Default	Size	Range	Example
boolean	stores true or false	false	1 bit	true or false	boolean v = true;
byte	stores whole numbers and is used for small values	0	8 bits	-2^7 to 2^7-1	byte a = 10;
char	stores a single Unicode character	\u0000	16 bits	'\u0000' to '\uffff'	char c = 'A';
short	stores whole numbers and is used for smaller values	0	16 bits	-2^{15} to $2^{15}-1$	short s = 10;
int	stores whole numbers and is the preferred data type for numeric values	0	32 bits	-2^{31} to $2^{31}-1$	int x = 1000;
long	stores whole numbers and is used for large values that int cannot store	0	64 bits	-2^{63} to $2^{63}-1$	long a = 100000L;
float	stores fractional numbers with 6-7 decimal digits	0.0	32 bits	1.4e-045 to 3.4e+038	float f = 23.5f;
double	stores fractional numbers with up to 15 decimal digits	0.0	64 bits	4.9e-324 to 1.8e+308	double d = 32.5;

Consider the example of coffee shop. You might have ordered different types of coffee like Caramel Frappuccino Mini, Caffe Latte Tall, Caffe Mocha Grande, etc. Just like how we have different sizes of coffee cups like mini, tall, grande etc., in Java, primitive data types have different sizes.



double



long



int



short

Here are some of the important points about primitive data types:

1. Numeric and boolean (true, false) values are written without quotes. E.g. `int score = 85;` `boolean isQualified = true;`
2. The character value must be written in single quotes while assigning it to a character variable. E.g. `char gender = 'M';`
3. A long value is assigned to the variable, suffixed with L (uppercase letter or lower case letter L can be used). E.g. `long salary = 500000L;`
4. A float value must be suffixed with F or f while assigning to the variable. E.g. `float average = 78.6f;`
5. You will learn about different places in the program where the variables can be declared. As of now, note that the default values are not applicable to the variables declared inside a method. The variables declared inside a method must be initialized with a value before printing their value or performing any operation on them. E.g.

```
class Demo {  
    public static void main(String[] args) {  
        int quantity;  
        float totalCost = 10 * quantity; // error on this line  
        System.out.println(totalCost);  
    }  
}
```

Non-primitive data types include classes, arrays, interfaces etc. These can be predefined in Java library or user-defined in the programs.

E.g. - **String** is a predefined class in Java library which is used to store a sequence of characters:

```
String customerName = "Jamie";
```

In Java, variable names should be nouns starting with lowercase letter. If it contains multiple words, then every inner word must start with capital letter. This type of casing is called camel casing.

Some examples of variables are given below:

- mobileNumber
- name
- unitPrice
- paymentMode
- age

Q1 of 5

Which of the following are valid identifiers?

abc

Student

class

public

1num

result

Q1 of 5

Which of the following are valid identifiers?

abc

Student

class

public

1num

result

Option abc is Correct

Option Student is Correct

Option result is Correct

Q2 of 5

Identifiers cannot start with _____.

character

digit

\$

Q2 of 5

Identifiers cannot start with _____.

character

digit

—
\$

Option digit is Correct

Q3 of 5

What will be the output of the below code?

```
public static void main(String args[]) {  
    int age;  
    System.out.println(age);  
}
```

0

garbage value

compilation error : variable needs to be initialized before use

2147483647



Q3 of 5

What will be the output of the below code?

```
public static void main(String args[]) {  
    int age;  
    System.out.println(age);  
}
```

0

garbage value

compilation error : variable needs to be initialized before use

2147483647

Option compilation error : variable needs to be initialized
before use is Correct

Q4 of 5

Which of the following are valid variable declarations?

float = 10.5f

boolean isOn = "true"

char gender = 'F'

int value = 2

Q4 of 5

Which of the following are valid variable declarations?

float = 10.5f

boolean isOn = "true"

char gender = 'F'

int value = 20

Option char gender = 'F' is Correct

Option int value = 20 is Correct

Q5 of 5

Which of these values can a boolean variable contain?

true

false

0

1

Q5 of 5

Which of these values can a boolean variable contain?

true

false

0

1

Option true is Correct

Option false is Correct

Thank you for your attempt.